



Search Relevance Built Into the Vector Index

Evgeniya Sukhodolskaya & Nathan LeRoy @ Qdrant

This Might Be Interesting to You

- **If you're a search engineer**
 - An overview of IR research subdomain (relevance feedback) and how search industry bridges the gap between academia and productional pipelines
- **If you use retrieval in your pipelines (e.g. RAG)**
 - Approaches for improving the relevance of retrieved results
- **If you're just curious**
 - The story of what happens when someone from industry goes looking for answers in academia



Search Relevance

Search

WomenMenKids

NEW INClothingShoesSportsStreetwearAccessoriesBeautyDesignerBrandsSale %Pre-owned

EN🌐👤❤️🛒

🔍 black summer dress

'Black Summer Dress' ×

WomenMenKids

Sort bySizeBrandPriceMaterialPatternFitShape

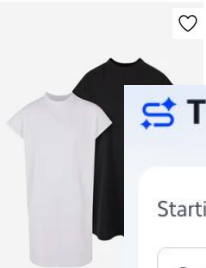
Product standardAdaptive FeaturesLengthShow all filters

4,673 items ⓘ



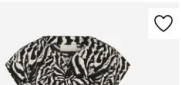
Deal

Sponsored ⓘ
Pepe Jeans
W11 MAYRA - Day dress - black
€78.00
Regular price: €190.00 -40%



Deal

Sponsored ⓘ
Urban Classics
PACK OF 2 - Jersey dress - I
€13.99
Originally: €34.99
Last lowest price: €15.49 -10%



Deal

Sponsored ⓘ
Urban Classics
PACK OF 2 - Jersey dress - I
€13.99
Originally: €34.99
Last lowest price: €15.49 -10%

Trip.Planner

Starting from Any ▾

Heading to

Country / City / Landmark

Date/Duration

Select

Preferences ▾

Create It Myself

✦ Plan a Trip with AI

Search Relevance

How fitting the results are to a request's intent, from the perspective of a search user

 The best Vector Search Engine?



Qdrant

<https://qdrant.tech>

Qdrant - Vector Search Engine

Qdrant helps you build the AI retrieval you want. Ship high performance, full-featured any scale and with any deployment model. Start Free ... [Read more](#)



Qdrant

'Black Summer Dress' ×

Women

Men

Kids

Sort by ▾ Size ▾ Brand ▾ Price ▾ Material ▾ Pattern ▾ Fit ▾ Shape ▾

Product standard ▾ Adaptive Features ▾ Length ▾ [Show all filters](#)

4,673 items 



Deal

Sponsored 

Pope Jeans
W11 MAYRA - Day dress - black
€78.00
Regular price: €139.99 -40%



Deal

Sponsored 

Urban Classics
PACK OF 2 - Jersey dress - black white
€13.99
Originally: €34.99
Last lowest price: €16.49 -10%



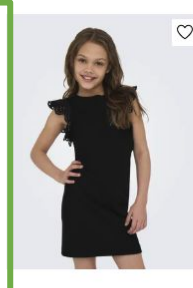
New

Sponsored 

Kaffe
KABELLA JERSEY - Jersey dress - black
deep chalk s animal
€59.95



Stradivarius
ASYMMETRIC - Shift dress - black
€22.99



NLY GIRLS
OGDREA LIFE S/S DRESS JRS NOOS -
jumper dress - black
from **€22.95**



 giulia_scotti + Follow

Search Can Miss Relevant Results

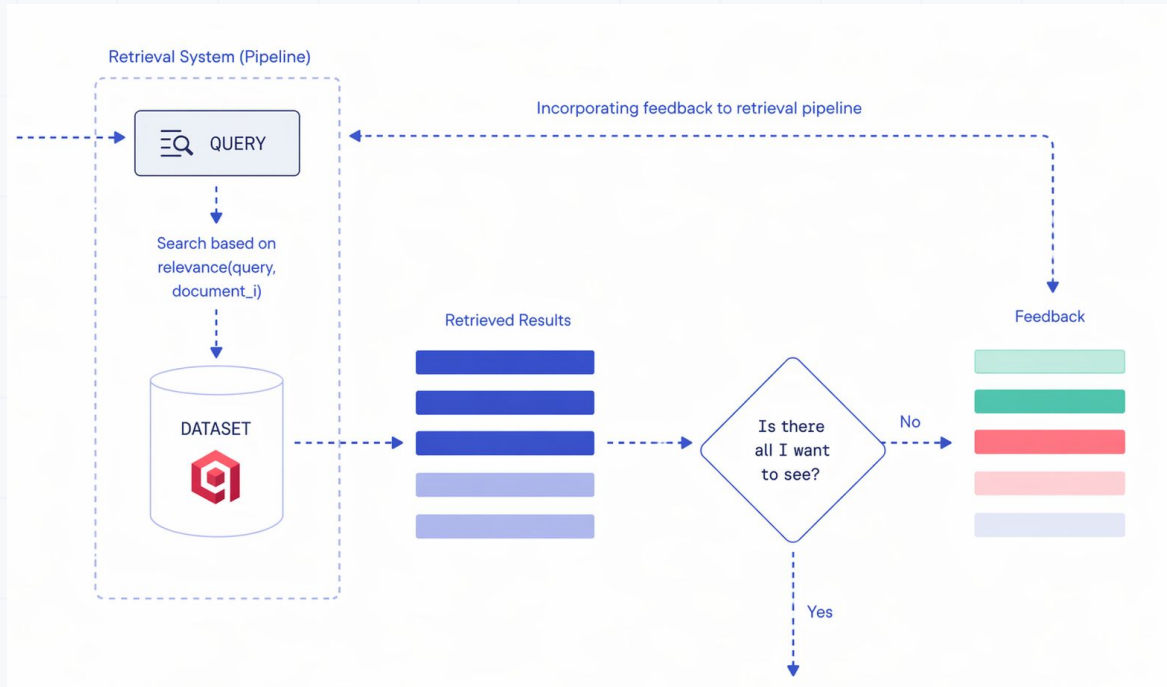
- Search algorithm is not good for your use case:
 - The way of measuring **search relevance score** between query and documents (we use this score to find & surface the best matches) is uncalibrated;
- Or it's generally ok, but doesn't reflect this user's understanding of relevance;
- The **search query** is not that good;
- The query is good, but not for this **dataset**: data is specific, skewed, etc...

...or just because it's a bad day for a search engineer:(



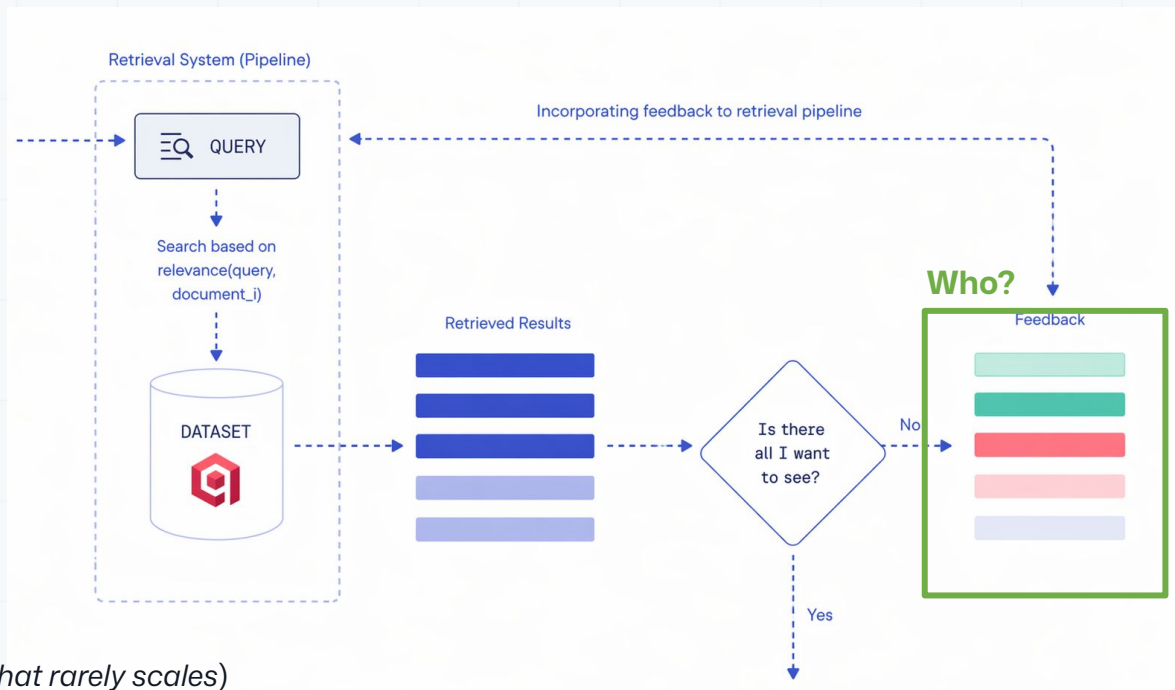
Improving Search Relevance: Relevance Feedback Approach

Feedback Loops in Search



Relevance Feedback – A mechanism for a retrieval system to **iteratively refine results in the direction of relevance**.

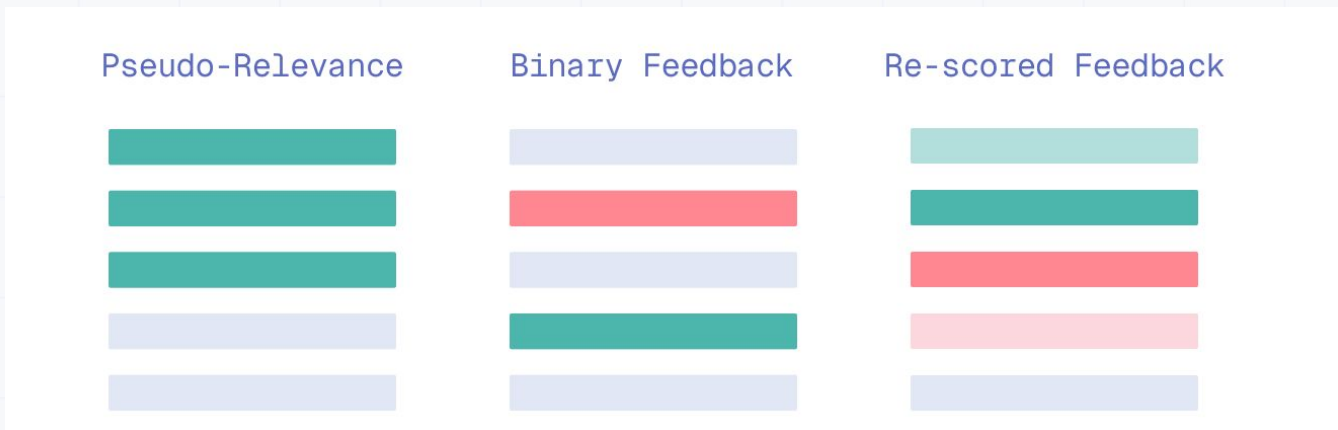
Who Provides Feedback?



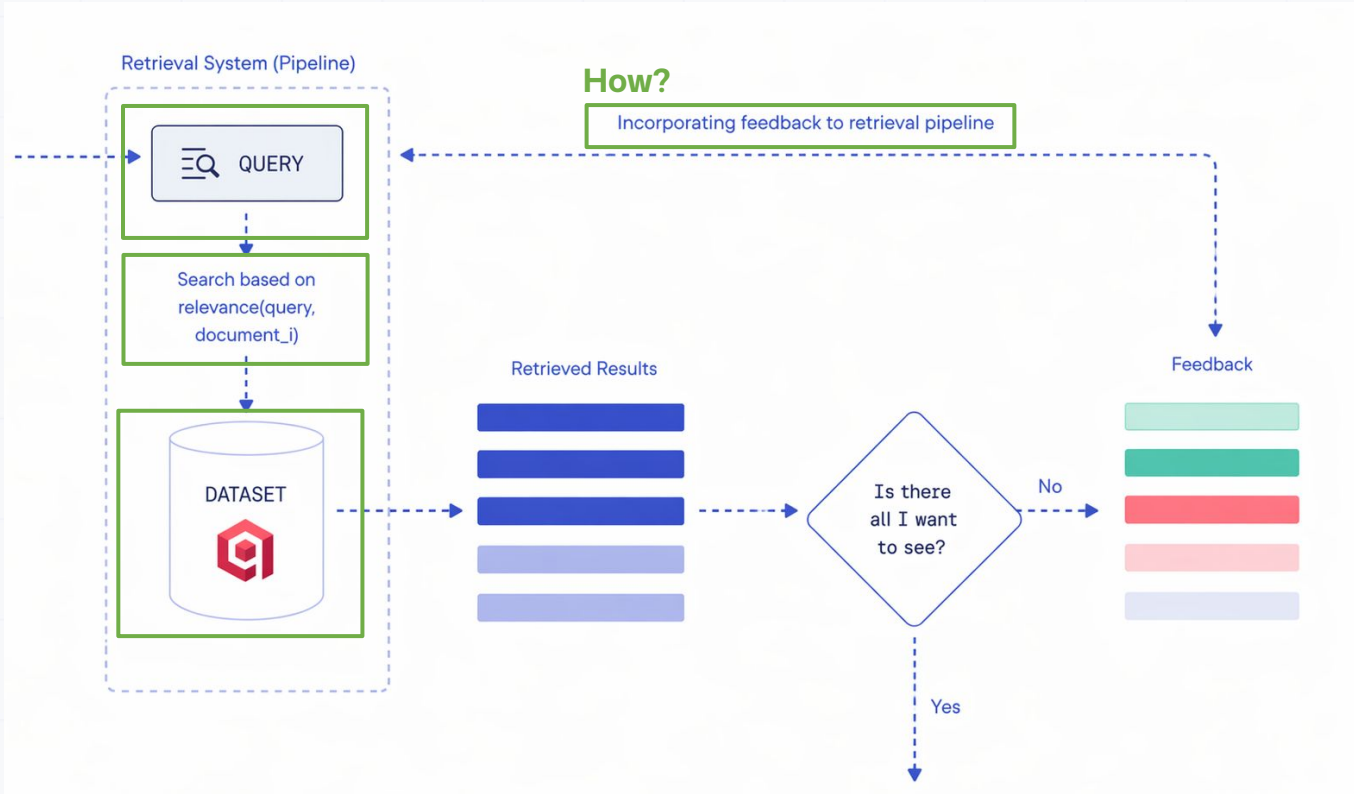
- **User** (*but that rarely scales*)
- In scalable systems, often a "**machine**": LLM (agent), reranker, heuristic/statistical function...

How Feedback Can Look?

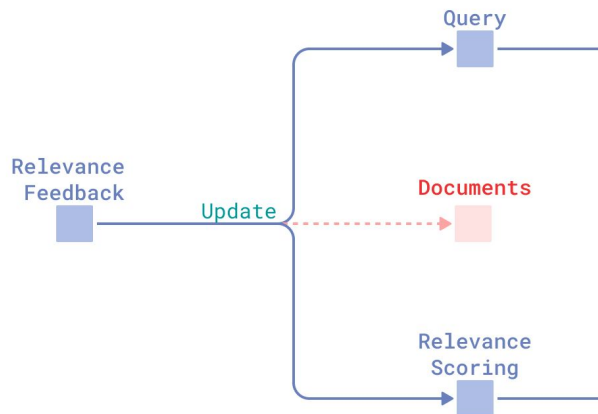
- **Pseudo-Relevance Feedback** – automatically assume top retrieved results are already relevant (*are they though?*)
- **Binary** – relevant / not relevant (*not always so easy in reality:/*)
- **Graded (re-scored)** – a relevance score, higher = more relevant, usually (*the best!*)



What to Do with Feedback?



What to Do with Feedback?



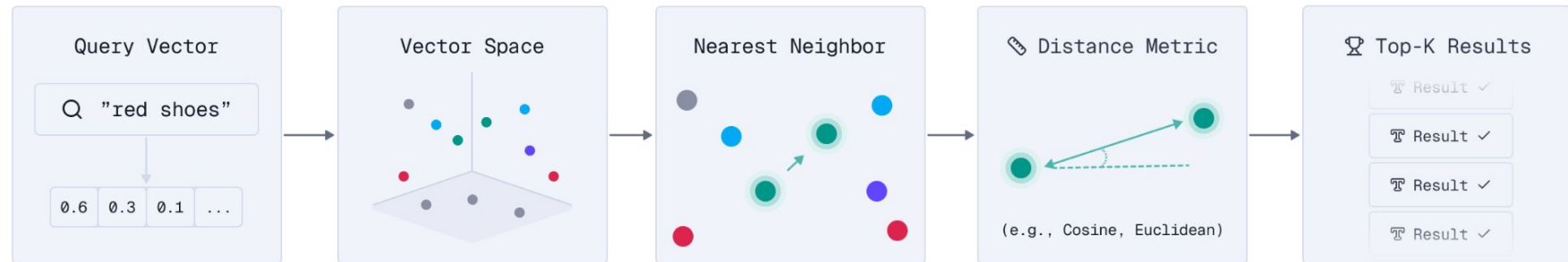
Goal: Relevance Feedback for Neural Search

Neural search works on vectors produced by ML models (embeddings).

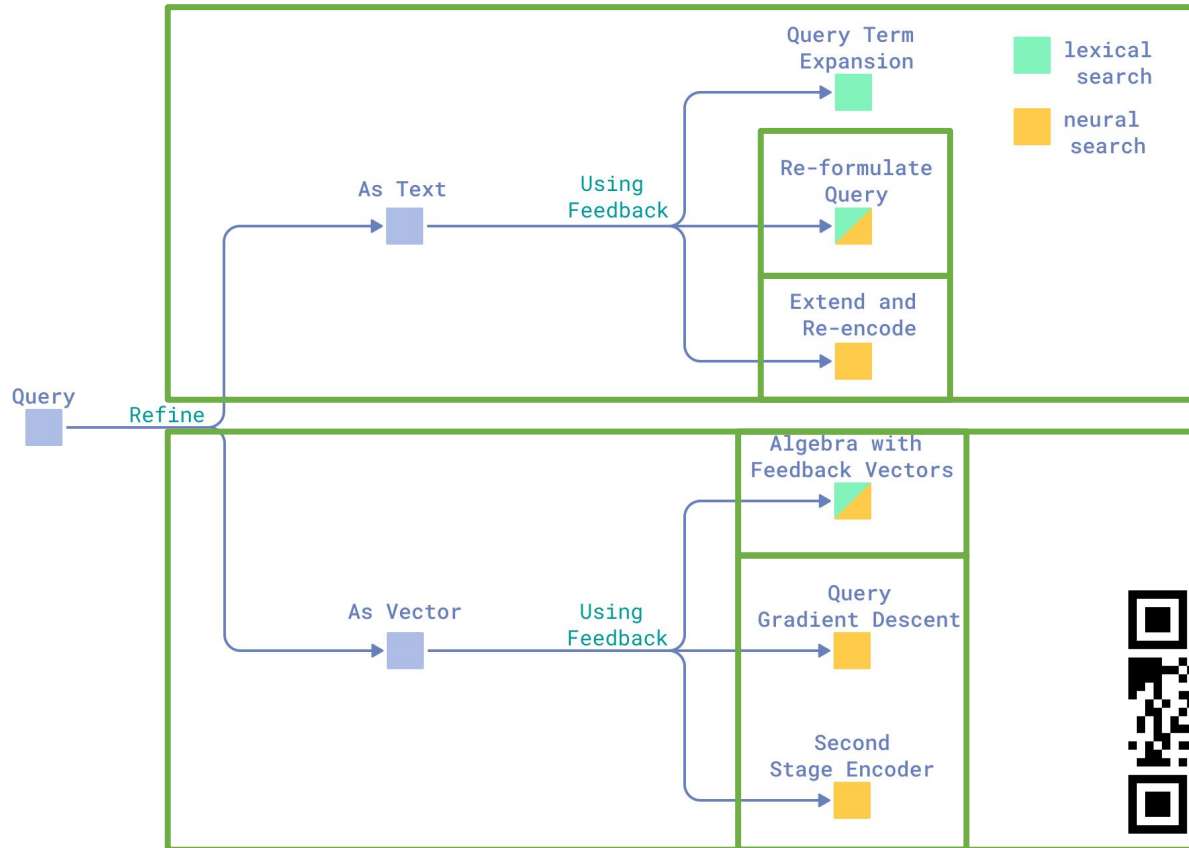
Often called **semantic similarity search**:

$F_{\text{relevance}}(\text{query}, \text{document}_i)$ is usually **$\text{cosine_similarity}(\text{query}, \text{document}_i)$** .

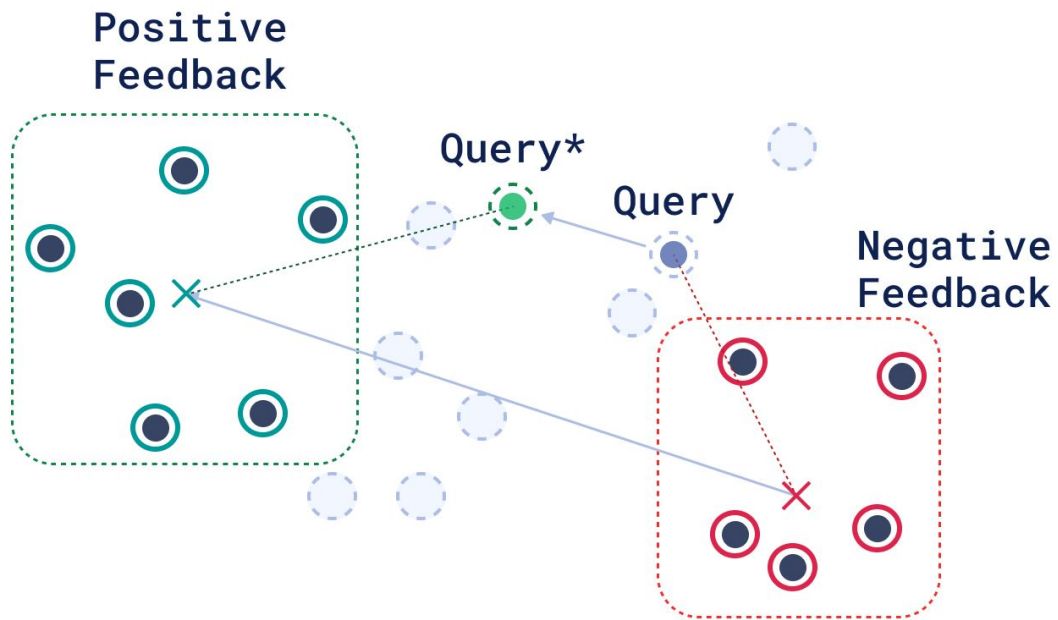
Needs vector index and vector search engine orchestrating retrieval.



What to Do with Feedback?

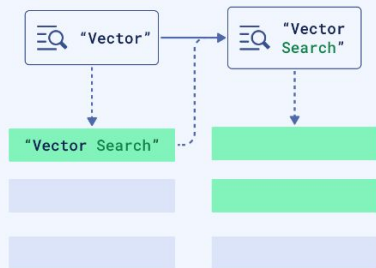


Modifying Query as a Vector

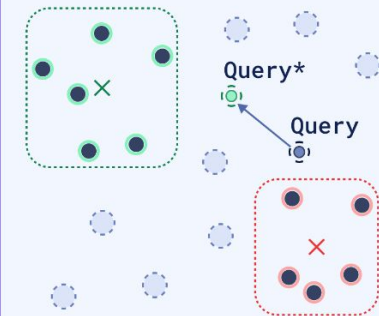


The Majority of the Methods = Query Modification

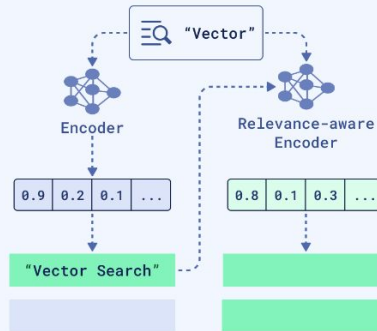
Query Expansion



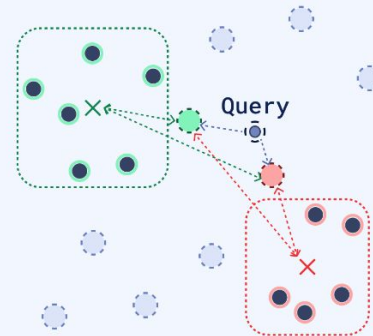
Algebra with a Query Vector



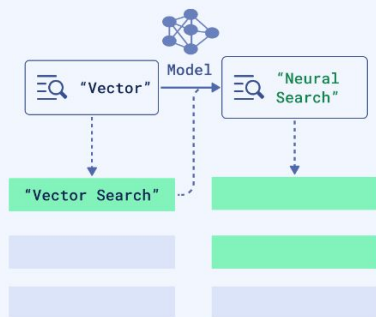
Extend and Re-encode Query



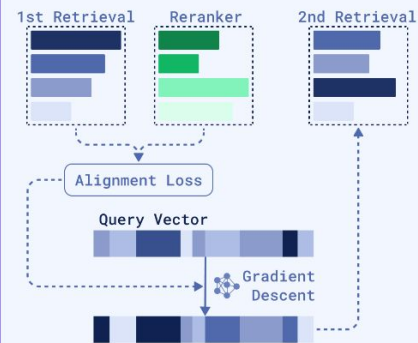
Combine Similarity Scores



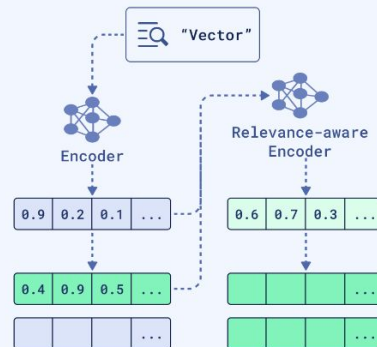
Query Reformulation



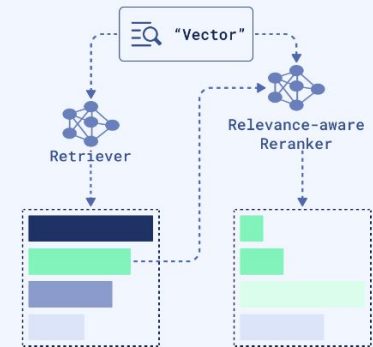
Query Vector Gradient Descent



Query Vector Re-encoder



Rescore with an Adapted Reranker

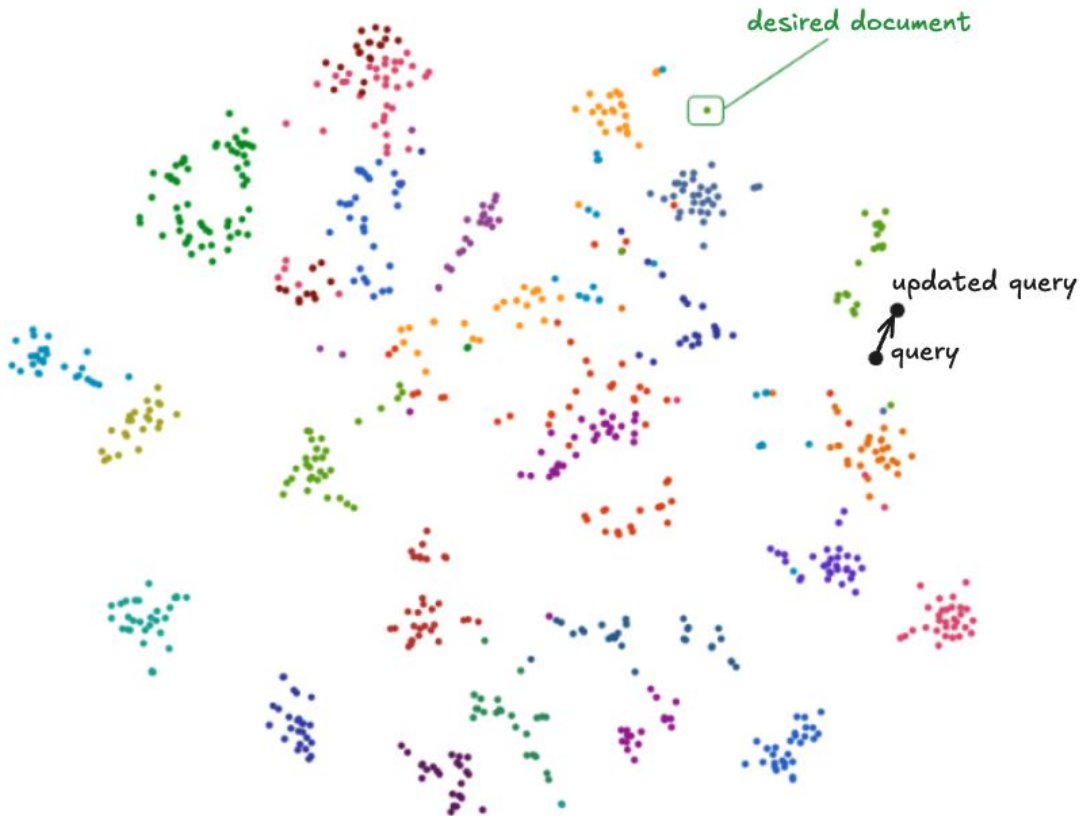


Why Wasn't Query Modification Interesting for Us?

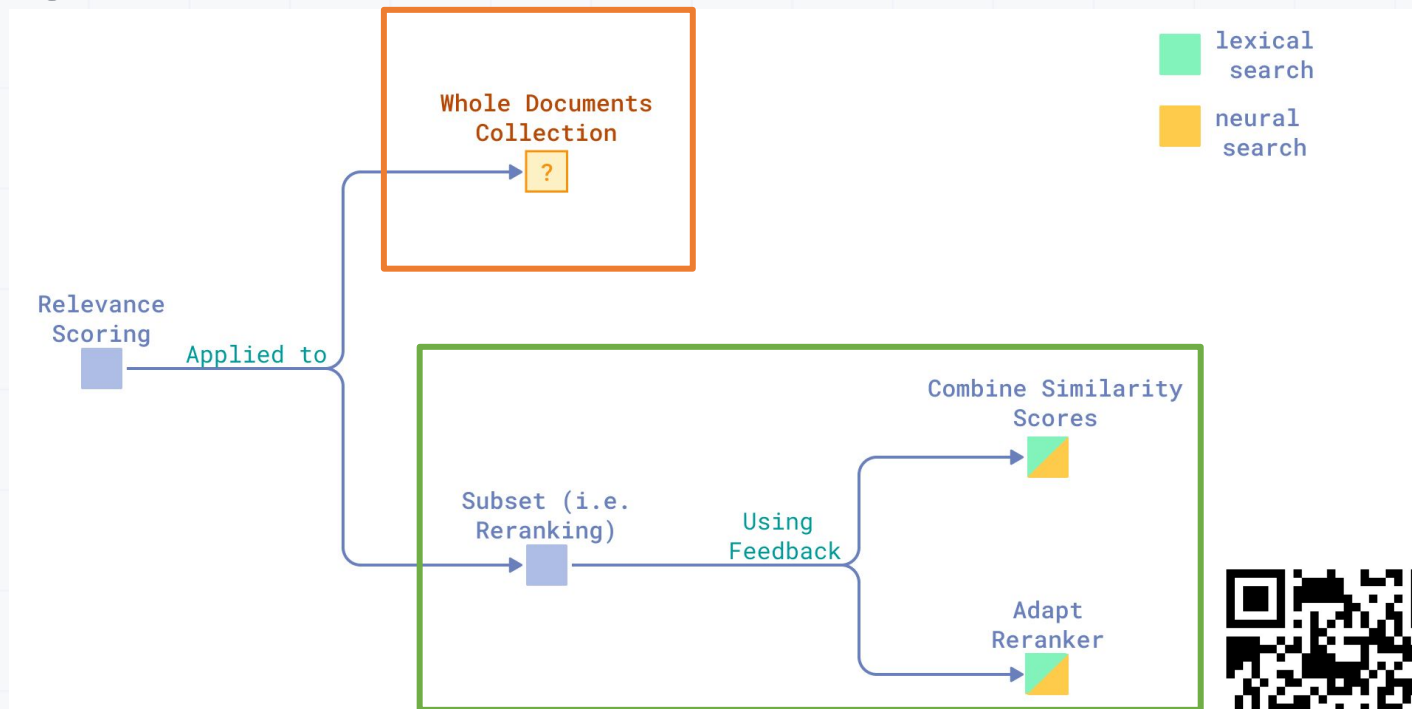
Query modification

happens on the
application side.

And **matching the
query to documents**
IS a **search engine**
concern.



What to do with Feedback? Change Relevance Scoring*



* Function scoring how well your query matches your results.

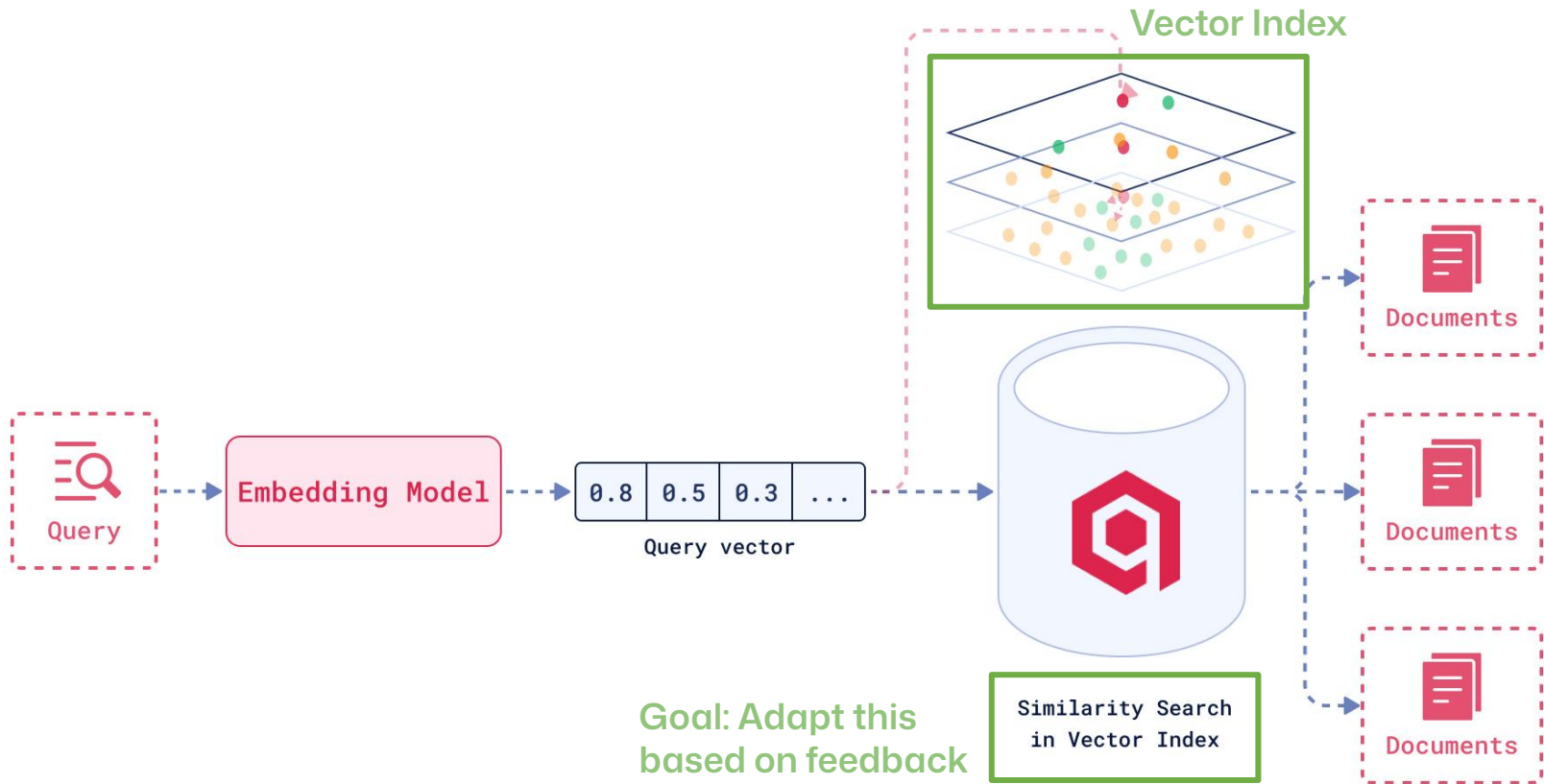




Vector Index-Native Relevance Feedback

***released in Qdrant 1.17.0**

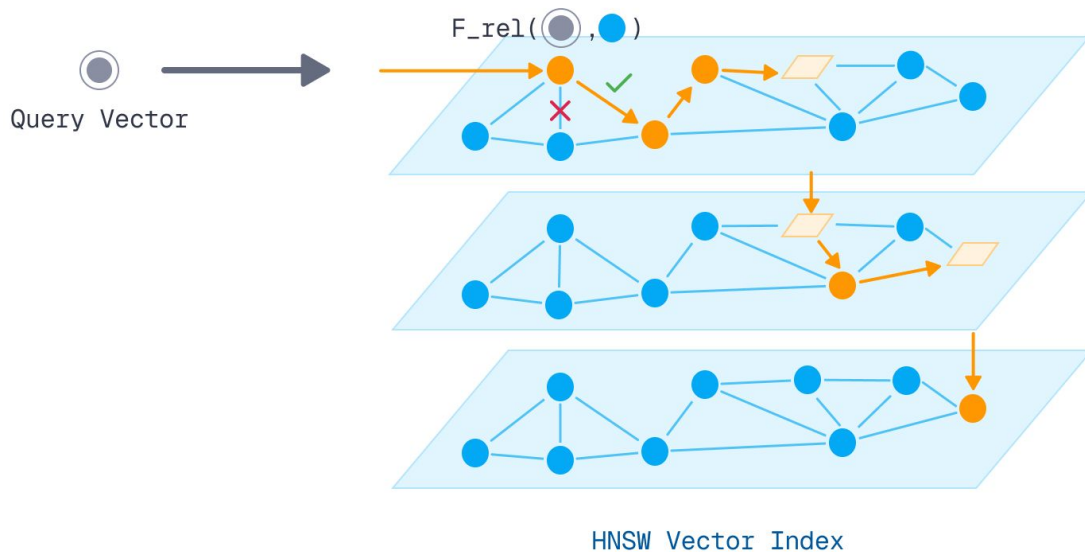
What Do I Mean by Vector Index-Native?



Similarity Search in Vector Index (HNSW)

Surfacing a document with the best **F_relevance score** found during vector index traversal

In neural search **F_relevance(query, document_i)** is usually cosine similarity



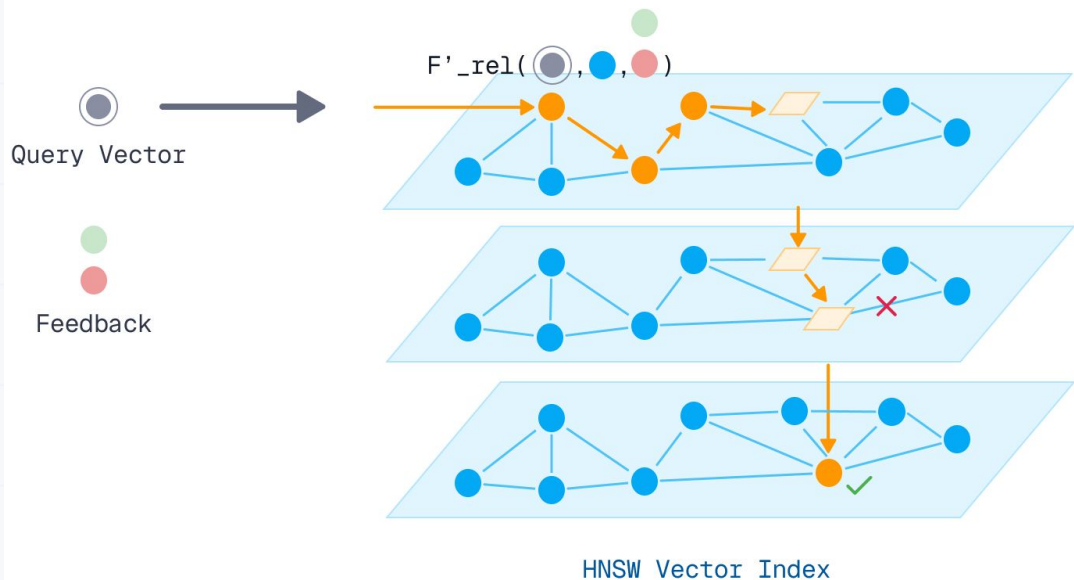
Let's Adjust $F_{\text{relevance}}(\text{query}, \text{document}_i)$!

What if we incorporated feedback **directly** into the $F_{\text{relevance}}$?

$F_{\text{relevance}}(\text{query}, \text{document}_i)$

→

$F'_{\text{relevance}}(\text{query}, \text{document}_i, \text{feedback})$



But How...?

How to **(1)** extract signals from feedback and **(2)** propagate them into relevance scoring, keeping it all:

- Cheap (Scalable)
- Universal => easily customizable to every:
 - Dataset;
 - Feedback model;
 - Embedding model used for retrieval.
- Applicable in production:
 - Even if retrieved results have nothing clearly relevant or irrelevant

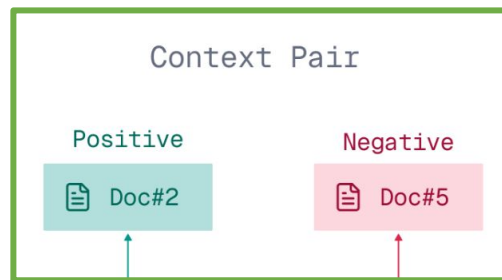
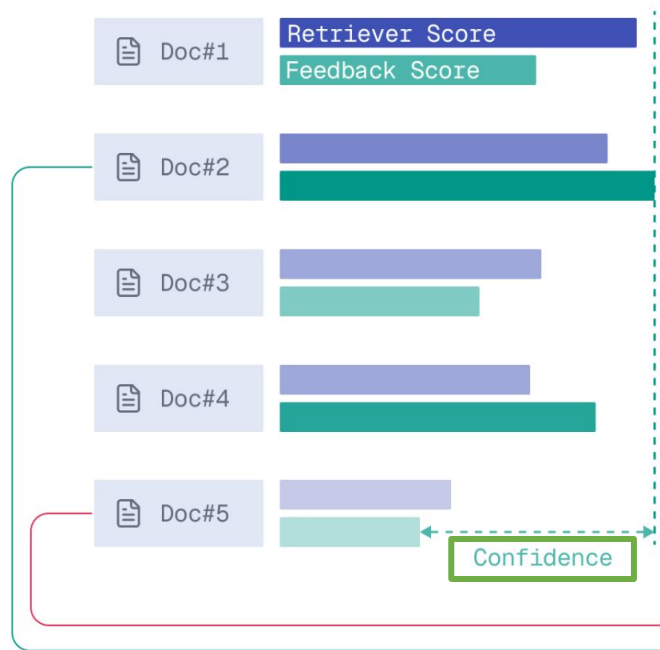
But How...?

How to **(1)** **extract signals from feedback** and **(2)** propagate them into relevance scoring, keeping it all:

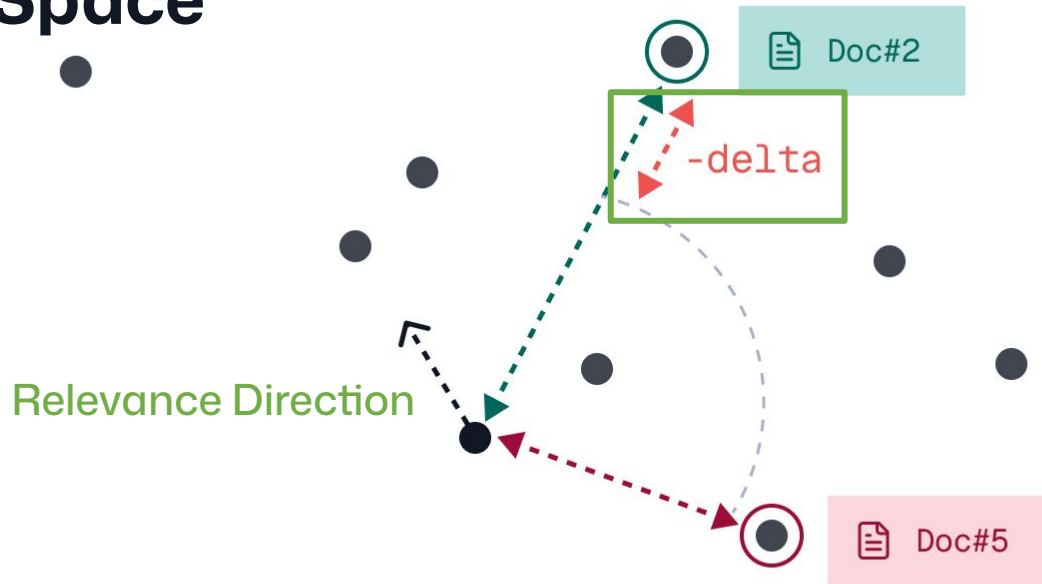
- Cheap (Scalable)
 - **Getting the most even with a little feedback;**
 - **No human labeling required** (*cause users won't provide it, trust me*)
- Universal => **easily customizable to every:**
 - Dataset;
 - **Feedback model;**
 - Embedding model used for retrieval.
- Applicable in production:
 - Even if **retrieved results have nothing clearly relevant or irrelevant**

Introducing Feedback Context Pairs

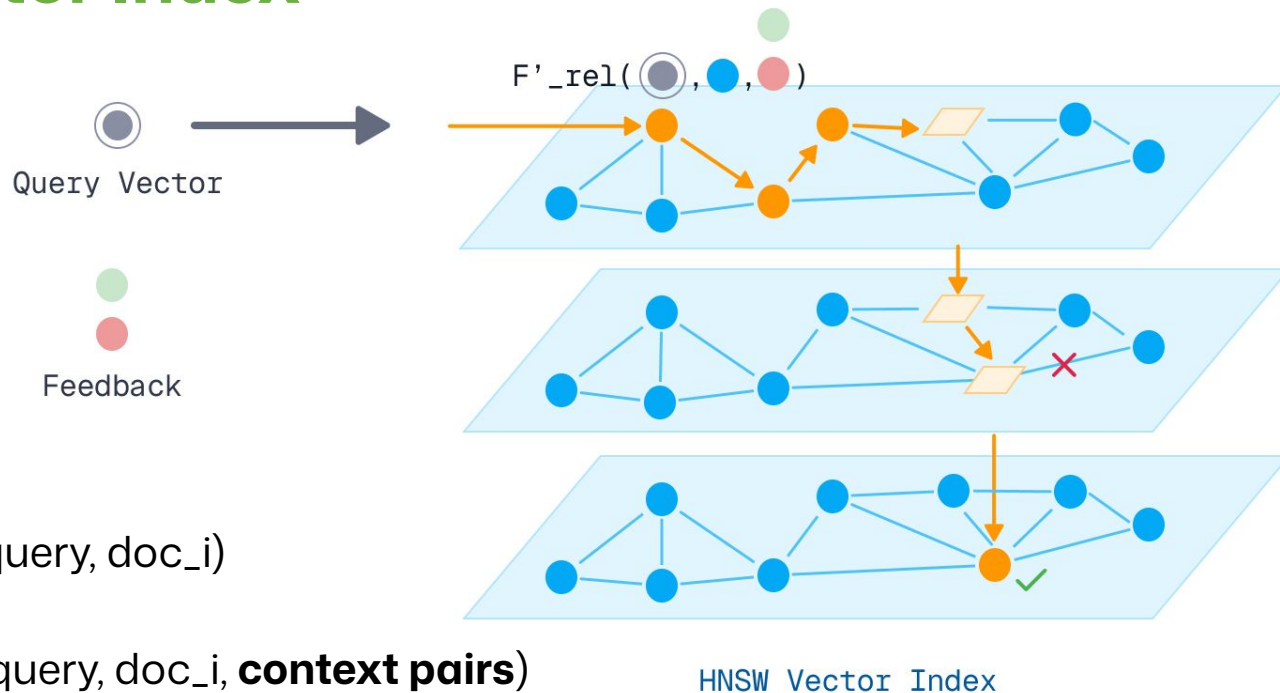
Re-score with Feedback Model



Context Pairs Provide Relevance Direction in Vector Space



Context Pairs Provide Relevance Direction in Vector Index



$F_{rel}(\text{query}, \text{doc}_i)$

→

$F'_{rel}(\text{query}, \text{doc}_i, \text{context pairs})$



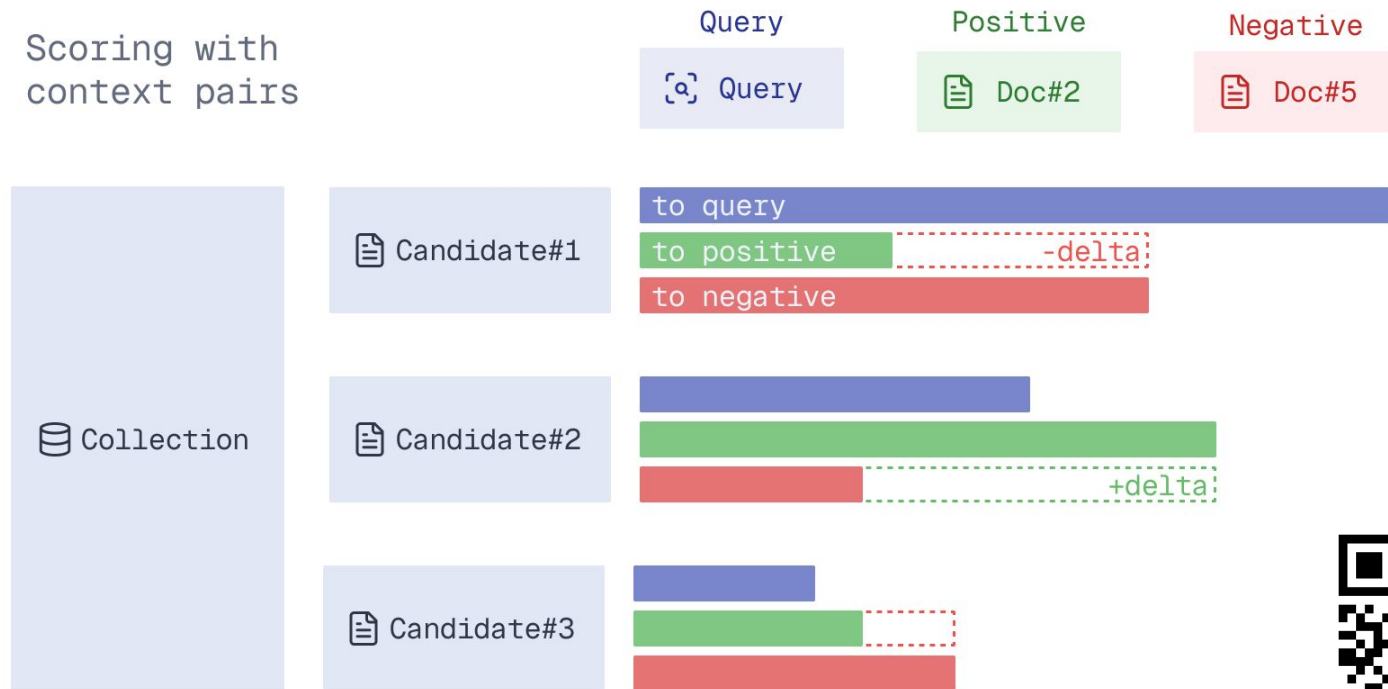
But How...?

How to **(1)** extract signals from feedback and **(2)** propagate them into relevance scoring, keeping it all:

- Cheap (Scalable)
 - Getting the most even with a little feedback;
 - No human labeling required (*cause users won't provide it, trust me*)
- Universal => easily customizable to every:
 - Dataset;
 - Feedback model;
 - Embedding model used for retrieval
- Applicable in production:
 - Even if retrieved results have nothing clearly relevant or irrelevant

F'_rel(query, doc_i, context pairs)

Scoring with
context pairs



F'_rel(query, doc_i, context pairs)

Scores distribution will differ per **retriever, feedback model and dataset**.

How to adapt scoring per triplet?

Pick some sensible formula, introduce parameters & obtain them through training.

$$F = a \cdot \cos(\text{query}, \text{doc}) + \sum_{p=1}^{\text{\#pairs}} \text{confidence}_p^b \cdot c \cdot \text{delta}_p(\text{doc})$$

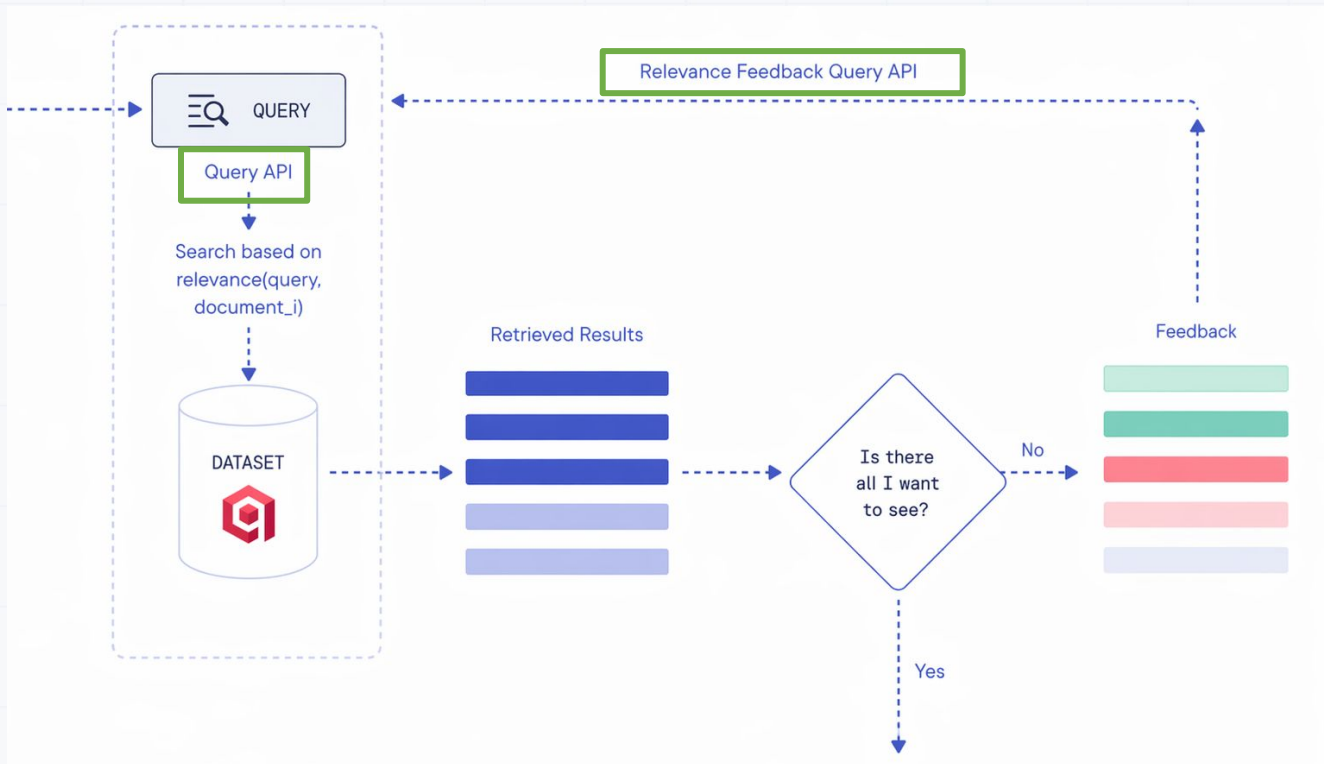
a, b, c – just 3 parameters, so 50–100 training examples are enough.





Relevance Feedback Query API

How Does it Work?



How to Use it?



```
POST /collections/{collection_name}/points/query
{
  "query": {
    "relevance_feedback": {
      "target": <query>,
      "feedback": [
        { "example": <doc_1>, "score": 0.68 },
        { "example": <doc_2>, "score": 0.72 },
        { "example": <doc_3>, "score": 0.61 }
      ],
      "strategy": {
        "naive": {
          "a": 0.12,
          "b": 0.43,
          "c": 0.03
        }
      }
    }
  }
}
```

<https://pypi.org/project/qdrant-relevance-feedback/>

Improving Semantic Search on Documentation

🔍 "recommendations API, how to use"?

Initial Retrieval Results	Additional Results from Relevance Feedback-based Retrieval
[1] Recommendation API [2] recommendation API <i><chunk duplicate>.</i> [3] So, even when the API is not called recommend, recommendation systems can also use this approach and adapt it for their specific use-cases.	[1] Recap of the old recommendations API [2] The result of this API contains one array per recommendation requests. [3] Deliver Better Recommendations with Qdrant's new API

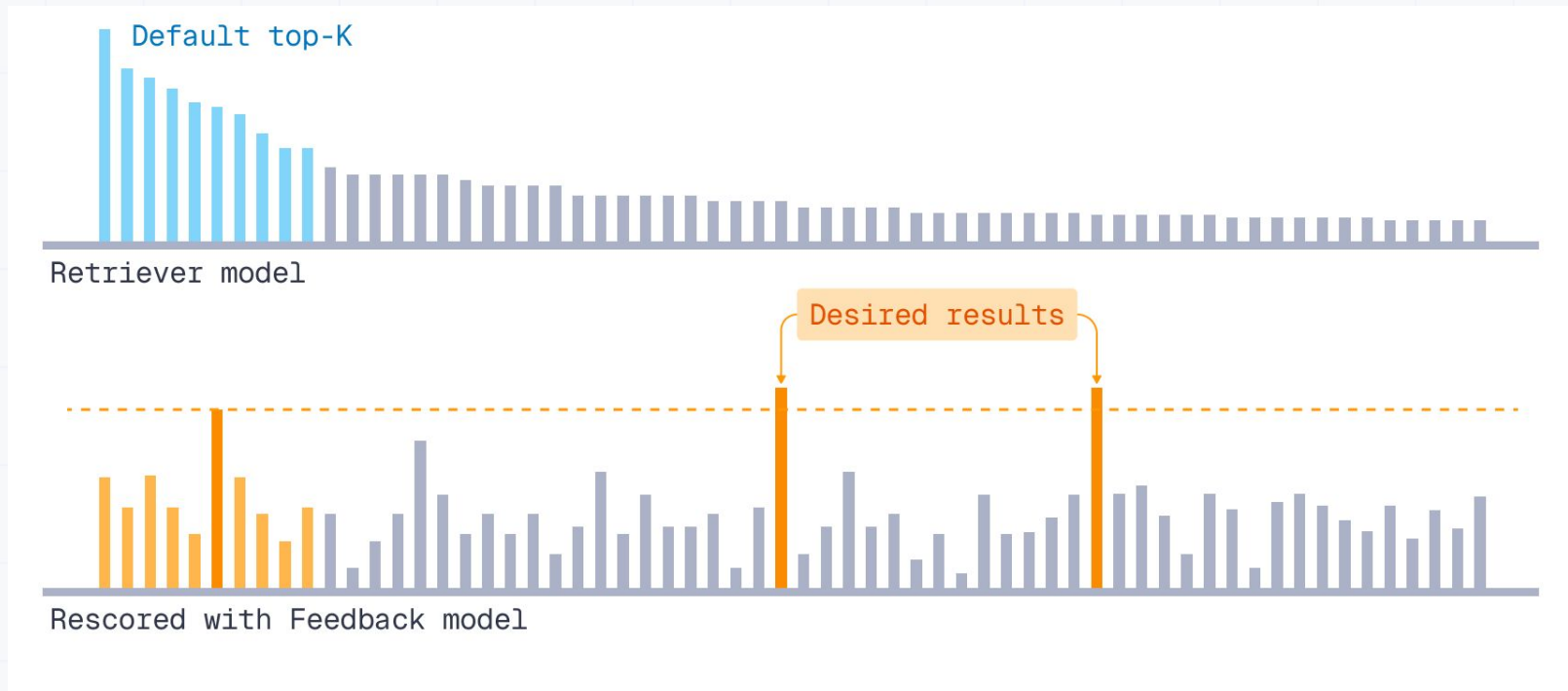


End-to-End Tutorial

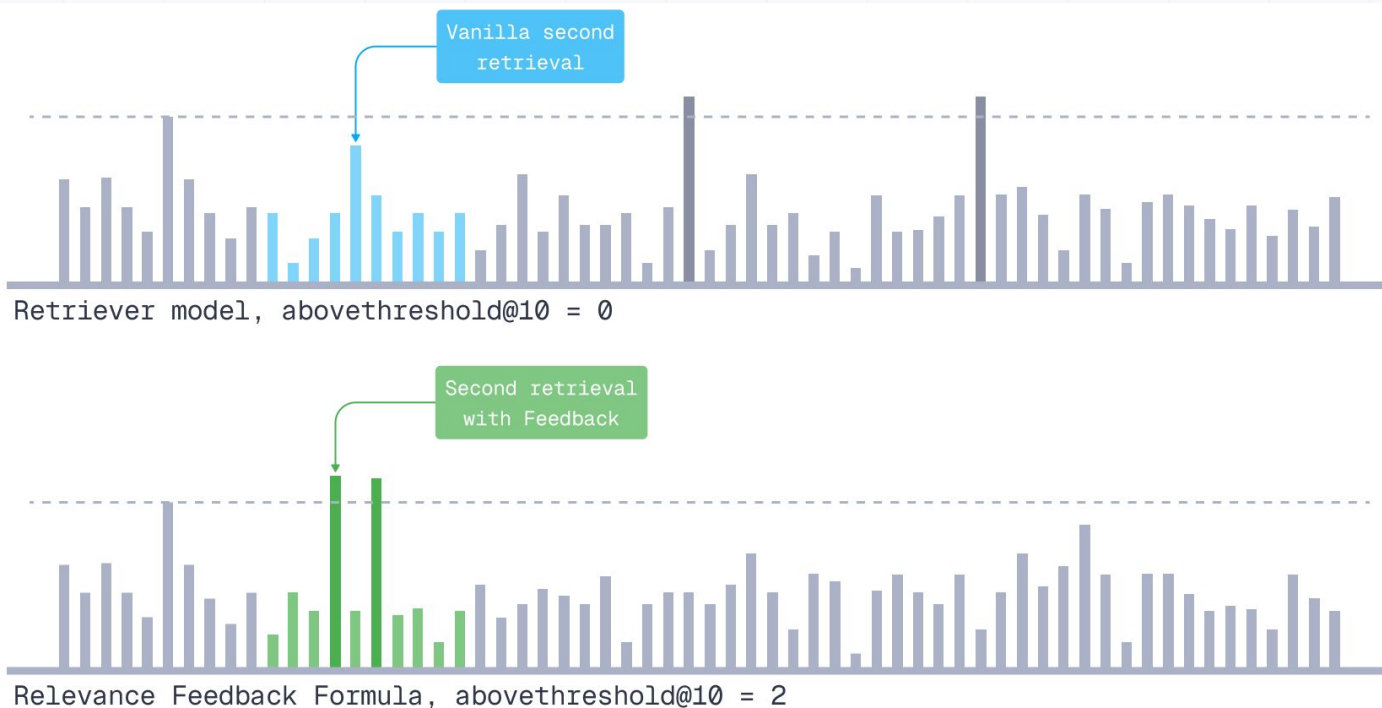


How to Evaluate

Relevance Gain per Query: Feedback Model is the Judge



Relevance Gain per Query



Q & A